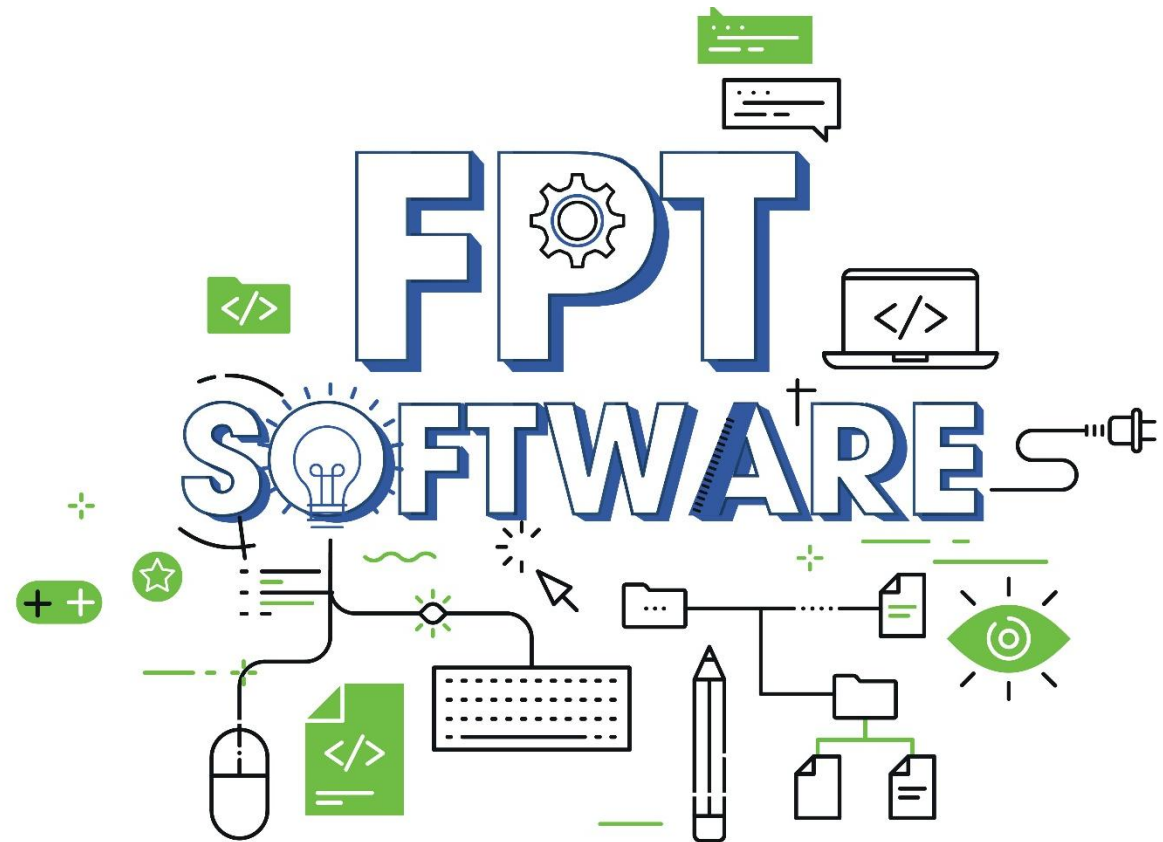


Android Advanced

Jetpack Compose



Agenda

1

- **What is Jetpack Compose**

2

- **Compose Components**

3

- **State in Compose**

4

- **Compose with libraries**



Lesson Objectives



Understanding Jetpack Compose

Implementing Compose components

Handling state in Compose

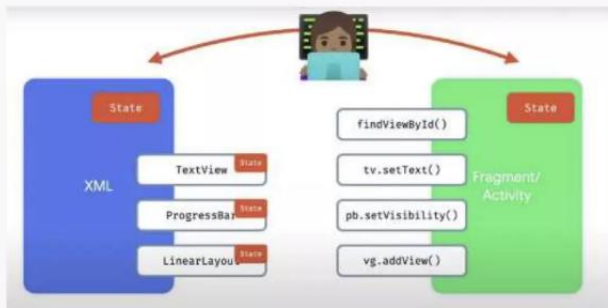
Implement compose with libraries

What is Jetpack Compose?

What is Jetpack Compose?

- Jetpack Compose simplifies and accelerates UI development on Android with less code, powerful tools, and intuitive Kotlin APIs.
- It's written completely in Kotlin and is fully compatible with the existing Android view system
- A new way of writing UI directly in code

Imperative UI



Google Developer Student Clubs

Declarative UI

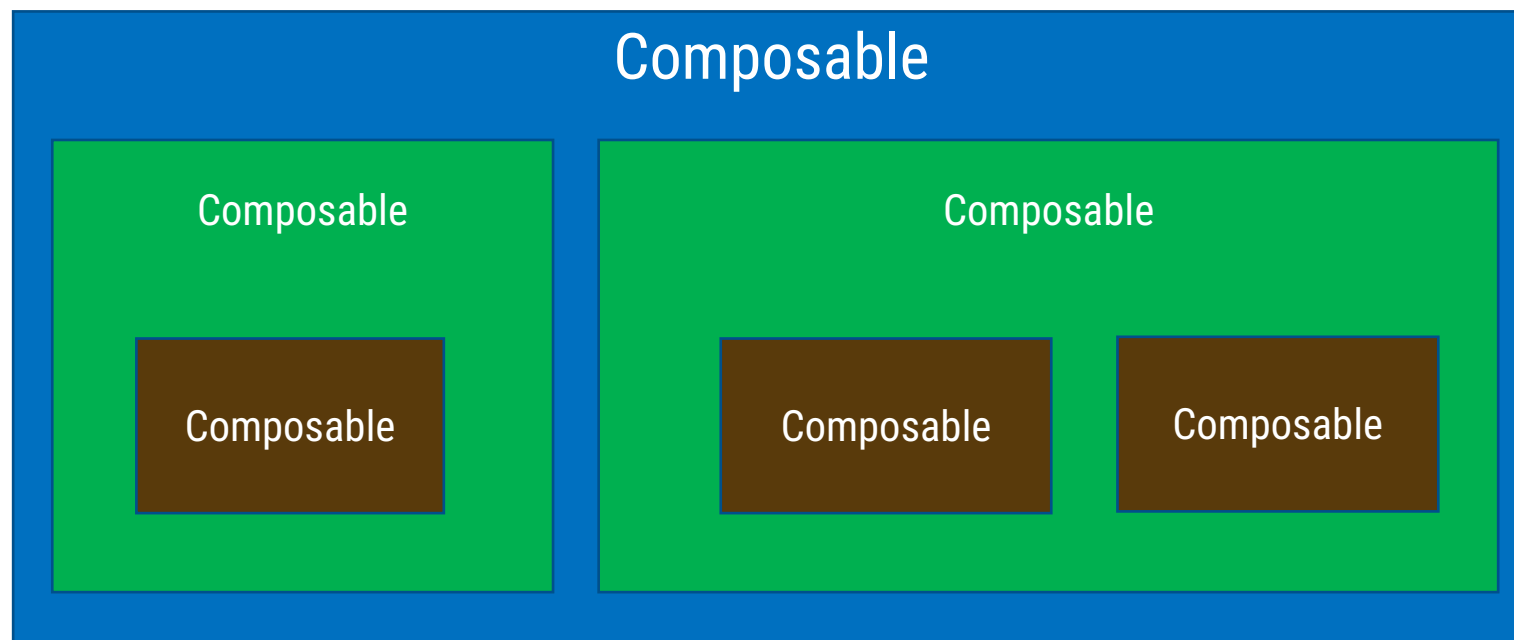


Why is it called Compose

- Old Android UI system → inheritance



- Compose UI system → **composition**

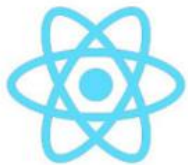


Benefits of using Jetpack Compose

- Declarative
- Compatible
- Increase development speed
- Concise and Idiomatic Kotlin
- Easy to maintain
- Written in Kotlin

Declarative UI story

13. 05



Declarative

React makes it painless to create interactive UIs. Design simple views for each state in your application, and React will efficiently update and render just the right components when your data changes.

Litho circled passing bintray v0.26.1 chat on glitter

Litho is a declarative framework for building efficient UIs on Android.

- **Declarative:** Litho uses a declarative API to define UI components for your UI based on a set of immutable inputs and the framework.
- **Asynchronous layout:** Litho can measure and layout your UI asynchronously on the UI thread.

17. 04



17. 05



Introduction to declarative UI

This introduction describes the conceptual difference between the declarative style used by Flutter, and the imperative style used by many other UI frameworks.

Declarative Syntax

SwiftUI uses a declarative syntax so you can simply state what your user interface should do. For example, you can write that you want a list of items consisting of text fields, then describe alignment, font, and color for each field. Your code is simple and easier to read than ever before, saving you time and maintenance.

```
import SwiftUI

struct ContentView : View {

    @State var model = Themes.listModel

    var body: some View {
        List(model.items, action: model.selectItem) { item in
```

19. 05



19. 06



📱 🌟

Jetpack Compose

A declarative toolkit for building UI

Jetpack Compose is an unbundled toolkit for building UI with the declarative programming model with the conciseness

extended

Thinking in Compose

Declarative

Focus on describing what you want the UI to look like, and the framework takes care of how to achieve that

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}

// In the UI code
Greeting(name = "OMAR")
```

Imperative

The traditional approach, requires developers to specify how the UI should be created and updated explicitly

```
class MainActivity : AppCompatActivity() {
    private lateinit var greetingTextView: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        greetingTextView = findViewById(R.id.greetingTextView)

        val name = "OMAR"
        greetingTextView.text = "Hello, $name!"
    }
}
```

Refer:

- [Imperative vs Declarative UI](#)
- [Thinking in compose](#)

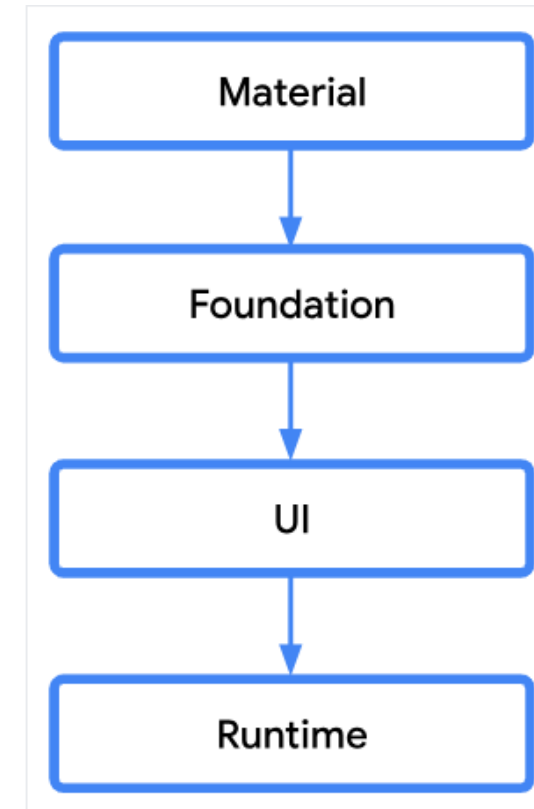
Jetpack compose Layers

Material: This module provides an implementation of the Material Design system for Compose UI, providing a theming system, styled components, ripple indications, icons. Build upon this layer when using Material Design in your app.

Foundation: This module provides design system agnostic building blocks for Compose UI, like [Row](#) and [Column](#), [LazyColumn](#), recognition of particular gestures, etc. You might consider building upon the foundation layer to create your own design system.

UI: The UI layer is made up of multiple modules ([ui-text](#), [ui-graphics](#), [ui-tooling](#), etc.). These modules implement the fundamentals of the UI toolkit, such as `LayoutNode`, [Modifier](#), input handlers, custom layouts, and drawing. You might consider building upon this layer if you only need fundamental concepts of a UI toolkit.

Runtime: This module provides the fundamentals of the Compose runtime such as [remember](#), [mutableStateOf](#), the [@Composable](#) annotation and [SideEffect](#). You might consider building directly upon this layer if you only need Compose's tree management abilities, not its UI.



Composable Functions

Kotlin

UI as a Function

Data → `fun Greeting(name: String) =
Text("Hello, $name")` → UI

```
@Composable
fun NewsFeed(stories: List<StoryData>, onSelected: (StoryData) -> Unit) {
    ScrollingList(stories) { story ->
        StoryWidget(story, onClick = { onSelected(story) })
    }
}

@Composable
fun StoryWidget(story: StoryData, onClick: () -> Unit) {
    Clickable(onClick) {
        Padding(8.dp) {
            Column {
                Title(story.title)
                Image(story.image)
                Text(story.content)
            }
        }
    }
}
```

Composable Functions



The three phases of a frame

1. **Composition:** What UI to show. Compose runs composable functions and creates a description of your UI.
2. **Layout:** Where to place UI. This phase consists of two steps: measurement and placement. Layout elements measure and place themselves and any child elements in 2D coordinates, for each node in the layout tree.
3. **Drawing:** How it renders. UI elements draw into a Canvas, usually a device screen.

Refer: [Compose phase](#)

Composition phase

In the composition phase, the Compose runtime executes composable functions and outputs a tree structure that represents your UI. This UI tree consists of layout nodes that contain all the information needed for the next phases

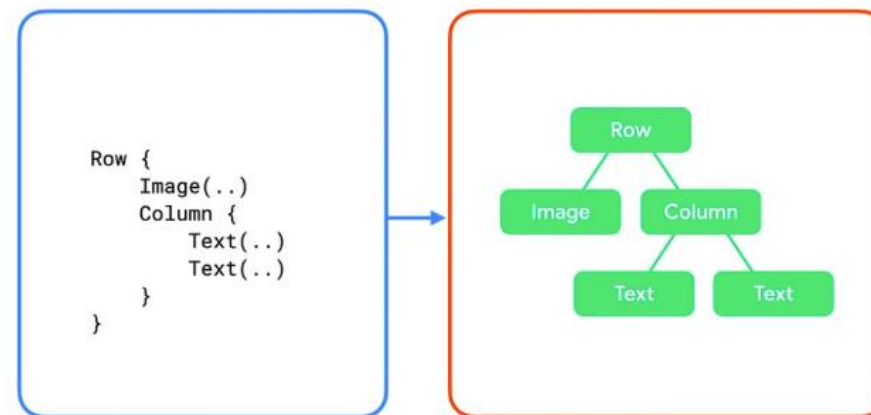
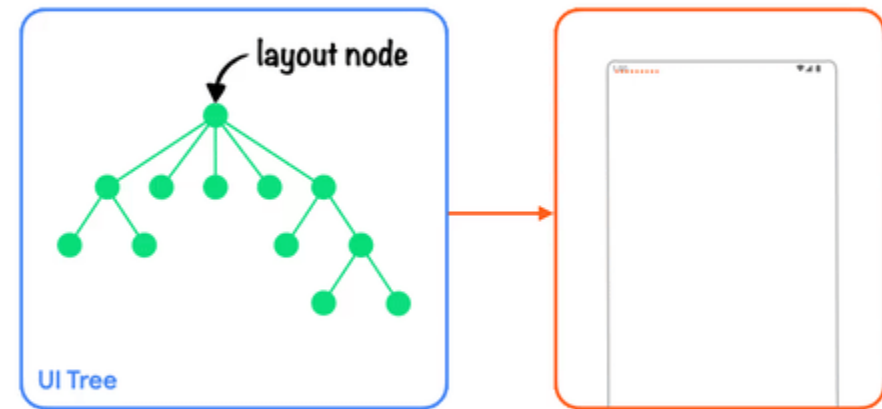


Figure 3. A subsection of a UI tree with the corresponding code.

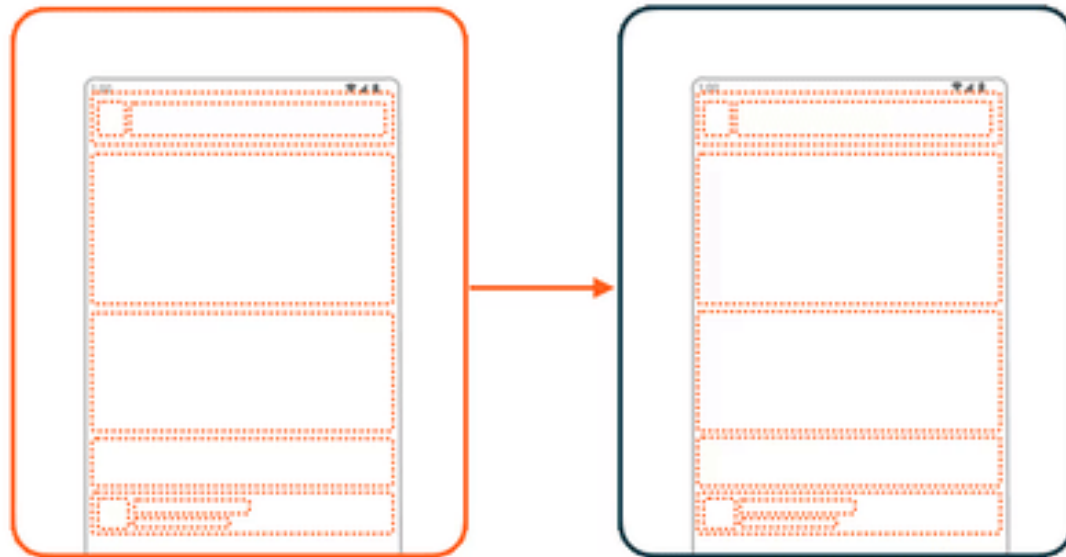
Layout phase

- In the layout phase, Compose uses the UI tree produced in the composition phase as input. The collection of layout nodes contain all the information needed to decide on each node's size and location in 2D space.
- During the layout phase, the tree is traversed using the following three step algorithm:
 1. **Measure children:** A node measures its children if any exist.
 2. **Decide own size:** Based on these measurements, a node decides on its own size.
 3. **Place children:** Each child node is placed relative to a node's own position.
- At the end of this phase, each layout node has:
 1. An assigned **width** and **height**
 2. An **x, y coordinate** where it should be drawn



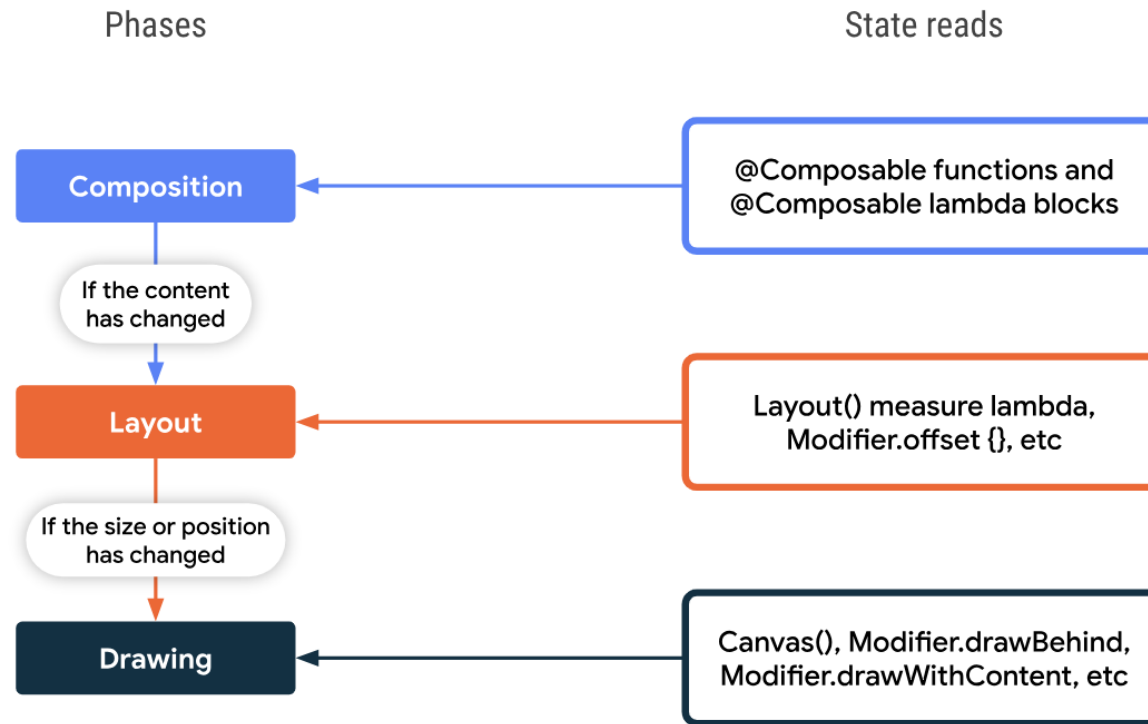
Drawing phase

- In the drawing phase, the tree is traversed again from top to bottom, and each node draws itself on the screen in turn.



Compose with state

- There are three main phases in Compose, and Compose tracks what state is read within each of them. This allows Compose to notify only the specific phases that need to perform work for each affected element of your UI.



More details: [Phased state reads](#)

Compose components

Compose components

Jetpack Compose offers an implementation of Material Design, a comprehensive design system for creating digital interfaces. You can use composable functions to implement Material components. It provides many basic components for UI development:

- Layout: Column, Row, Box, Constraints Layout, Modifier...
- View: Text, Text Field, Image, Button, Lists and Grids, Switch, Card, Chip, Pager
- Other: Scaffold, App bars, Navigation Drawer, Bottom Sheet, Flow Layouts
- Theme: Material Theme, Font, Emoji

Text composable

Display text

- The most basic way to display text is to use the Text composable with a String as an argument:

```
@Composable
fun SimpleText() {
    Text("Hello World")
}
```

Display text from resource

- Recommend you use string resources instead of hardcoding Text values

```
@Composable
fun StringResourceText() {
    Text(stringResource(R.string.hello_world))
}
```

Style text

The Text composable has multiple optional parameters to style its content. Below, we've listed parameters that cover the most common use cases with text.

Change text color

```
@Composable
fun BlueText() {
    Text("Hello World", color = Color.Blue)
}
```

Hello World

```
@Composable
fun TextShadow() {
    val offset = Offset(5.0f, 10.0f)
    Text(
        text = "Hello world!",
        style = TextStyle(
            fontSize = 24.sp,
            shadow = Shadow(
                color = Color.Blue, offset = offset, blurRadius = 8f
            )
        )
    )
}
```

Hello world!

```
@Composable
fun ItalicText() {
    Text("Hello World", fontStyle = FontStyle.Italic)
}
```

Hello World

```
@Composable
fun BoldText() {
    Text("Hello World", fontWeight = FontWeight.Bold)
}
```

Hello World

Change text size

```
@Composable
fun BigText() {
    Text("Hello World", fontSize = 30.sp)
}
```

Hello World

Multiple style with AnnotatedString

To set different styles within the same `Text` composable, use an `AnnotatedString`. `AnnotatedString` is a data class containing:

- A Text value
- A List of `SpanStyleRange`, It includes text properties such as color, font style, size, and other properties.
- A List of `ParagraphStyleRange`, specifying text alignment, text direction, line height, and text indent styling

```
@Composable
fun MultipleStylesInText() {
    Text(
        buildAnnotatedString {
            withStyle(style = SpanStyle(color = Color.Blue)) {
                append("H")
            }
            append("ello ")

            withStyle(style = SpanStyle(fontWeight = FontWeight.Bold, color = Color.Red)) {
                append("W")
            }
            append("orld")
        }
    )
}
```

TextSnippets.kt

Hello World

Advance styling with Brush

In Jetpack Compose, Brushes are used to define how colors or images will be applied to areas in the user interface.

```
val gradientColors = listOf(Cyan, LightBlue, Purple /*...*/)

Text(
    text = text,
    style = TextStyle(
        brush = Brush.linearGradient(
            colors = gradientColors
        )
    )
)
```

Do not allow people to
dim your shine
because they are
blinded. Tell them to
put some sunglasses
on.

TextField

TextField allows users to enter and modify text, There are two levels of TextField implementation:

- TextField is the Material Design implementation. We recommend you choose this implementation as it follows Material Design guidelines:
 - ✓ Default styling is filled
 - ✓ OutlinedTextField is the outline styling version
- BasicTextField enables users to edit text via hardware or software keyboard, but provides no decorations like hint or placeholder.

```
@Composable
fun SimpleFilledTextFieldSample() {
    var text by remember { mutableStateOf("Hello") }

    TextField(
        value = text,
        onValueChange = { text = it },
        label = { Text("Label") }
    )
}
```

TextSnippets.kt

Label
Hello

```
@Composable
fun SimpleOutlinedTextFieldSample() {
    var text by remember { mutableStateOf("") }

    OutlinedTextField(
        value = text,
        onValueChange = { text = it },
        label = { Text("Label") }
    )
}
```

TextSnippets.kt

Label
Hello Compose

Refer: [user-input](#)

Button

Buttons are fundamental components that allow the user to trigger a defined action. There are five types of buttons

Type	Appearance	Purpose
Filled	Solid background with contrasting text.	High-emphasis buttons. These are for primary actions in an application, such as "submit" and "save." The shadow effect emphasizes the button's importance.
Filled tonal	Background color varies to match the surface.	Also for primary or significant actions. Filled buttons provide more visual weight and suit functions such as "add to cart" and "Sign in."
Elevated	Stands out by having a shadow.	Fits a similar role to tonal buttons. Increase elevation to cause the button to appear even more prominently.
Outlined	Features a border with no fill.	Medium-emphasis buttons, containing actions that are important but not primary. They pair well with other buttons to indicate alternative, secondary actions like "Cancel" or "Back."
Text	Displays text with no background or border.	Low-emphasis buttons, ideal for less critical actions such as navigational links, or secondary functions like "Learn More" or "View details."



Button

API surface

- `onClick`: The function called when the user presses the button.
- `enabled`: When false, this parameter causes the button to appear unavailable and inactive.
- `colors`: An instance of `ButtonColors` that determines the colors used in the button.
- `contentPadding`: The padding within the button.

```
@Composable
fun FilledButtonExample(onClick: () -> Unit) {
    Button(onClick = { onClick() }) {
        Text("Filled")
    }
}
```

Button.kt

★ **Note:** If you would like to build a custom button, use the `Button` composable.

This implementation appears as follows:



Figure 2. A filled button.

```
@Composable
fun Button(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    shape: Shape = ButtonDefaults.shape,
    colors: ButtonColors = ButtonDefaults.buttonColors(),
    elevation: ButtonElevation? = ButtonDefaults.buttonElevation(),
    border: BorderStroke? = null,
    contentPadding: PaddingValues = ButtonDefaults.ContentPadding,
    interactionSource: MutableInteractionSource = remember { MutableInteractionSource() },
    content: @Composable RowScope() -> Unit
) {
```

Params:

- `onClick` - called when this button is clicked
- `modifier` - the `Modifier` to be applied to this button
- `enabled` - controls the enabled state of this button. When false, this component will not respond to user input, and it will appear visually disabled and disabled to accessibility services.
- `shape` - defines the shape of this button's container, border (when `border` is not null), and shadow (when using `elevation`)
- `colors` - `ButtonColors` that will be used to resolve the colors for this button in different states. See `ButtonDefaults.buttonColors`.
- `elevation` - `ButtonElevation` used to resolve the elevation for this button in different states. This controls the size of the shadow below the button. Additionally, when the container color is `ColorScheme.surface`, this controls the amount of primary color applied as an overlay. See `ButtonElevation.shadowElevation` and `ButtonElevation.tonalElevation`.
- `border` - the border to draw around the container of this button
- `contentPadding` - the spacing values to apply internally between the container and the content
- `interactionSource` - the `MutableInteractionSource` representing the stream of `Interactions` for this button. You can create and pass in your own remembered instance to observe `Interactions` and customize the appearance / behavior of this button in different states.

Image

- Use the Image composable to display a graphic on screen. To load an image (for example: PNG, JPEG, WEBP) or vector resource from the disk, use the painterResource API with your image reference

```
Image(  
    painter = painterResource(id = R.drawable.dog),  
    contentDescription = stringResource(id = R.string.dog_content_description)  
)
```

LoadingImagesSnippets.kt

- To load an image from the internet, there are several third-party libraries available to help you handle the process. Image loading libraries do a lot of the heavy lifting for you; they handle both caching (so you don't download the image multiple times) and networking logic to download the image and display it on screen

```
AsyncImage(  
    model = "https://example.com/image.jpg",  
    contentDescription = "Translated description of what the image contains"  
)
```

LoadingImagesSnippets.kt

Refer: [images loading](#)



The Icon composable is a convenient way to draw a single color icon on screen that follows Material Design guidelines

The [Material Icon library](#) also includes a set of predefined `Icons` that can be used in Compose without needing to import an SVG manually. To draw the rounded version of the [shopping cart icon](#) [↗](#):

```
Icon(  
    Icons.Rounded.ShoppingCart,  
    contentDescription = stringResource(id = R.string.shopping_cart_content_desc)  
)
```

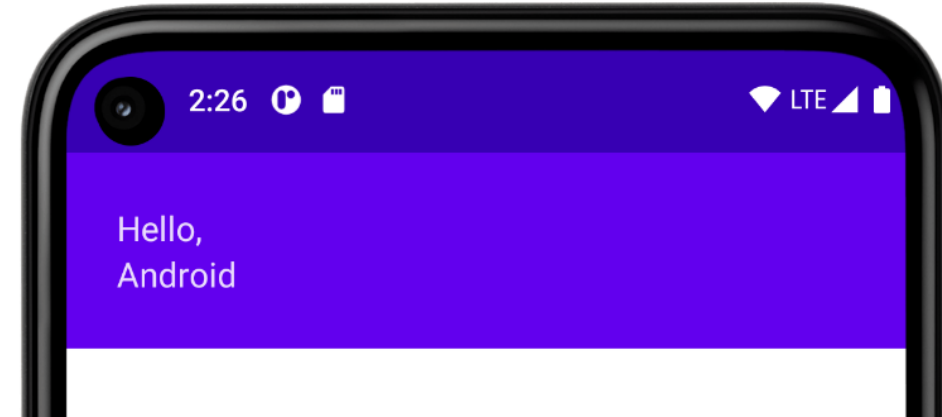
MaterialIconsSnippets.kt [↗](#)



Figure 1: Shopping cart vector with Icon

Modifier allow you to decorate or augment a composable, Modifier let you do these sorts of things:

- Change the composable's size, layout, behavior and appearance.
- Add information, like accessibility labels
- Process user input
- Add high-level interactions, like making an element clickable, scrollable, draggable or zoomable...



```
@Composable
private fun Greeting(name: String) {
    Column(
        modifier = Modifier
            .padding(24.dp)
            .fillMaxWidth()
    ) {
        Text(text = "Hello,")
        Text(text = name)
    }
}
```

Refer: [Compose modifier](#)

Modifier

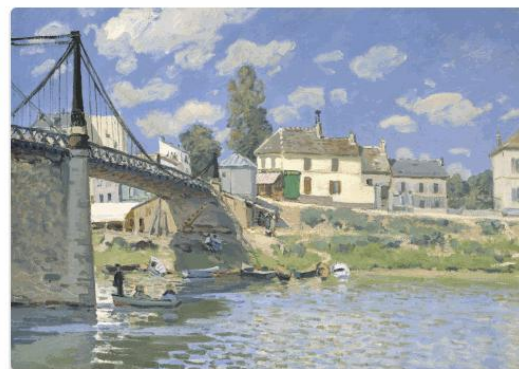
The order of modifier functions is significant. Since each function makes changes to the Modifier returned by the previous function, the sequence affects the final result

```
@Composable
fun ArtistCard(/*...*/) {
    val padding = 16.dp
    Column(
        Modifier
            .clickable(onClick = onClick)
            .padding(padding)
            .fillMaxWidth()
    ) {
        // rest of the implementation
    }
}
```

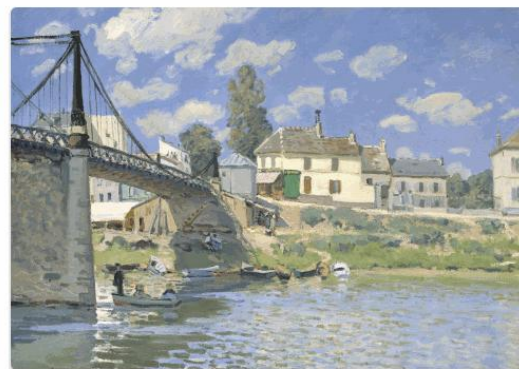
```
@Composable
fun ArtistCard(/*...*/) {
    val padding = 16.dp
    Column(
        Modifier
            .padding(padding)
            .clickable(onClick = onClick)
            .fillMaxWidth()
    ) {
        // rest of the implementation
    }
}
```



Alfred Sisley
3 minutes ago



Alfred Sisley
3 minutes ago



Modifier

Jetpack Compose provides a list of built-in modifiers to help you decorate or augment a composable.

- **Padding and size:** By default, layouts provided in Compose wrap their children. However, you can set a size by using the size modifier.
- **Offset:** To position a layout relative to its original position, add the offset modifier and set the offset in the x and y axis. Offsets can be positive as well as non-positive. The difference between padding and offset is that adding an offset to a composable does not change its measurements

[List of Compose modifiers](#) | [Jetpack Compose](#) | [Android Developers](#)

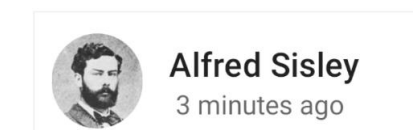
```
@Composable
fun ArtistCard(/*...*/) {
    Row(
        modifier = Modifier.size(width = 400.dp, height = 100.dp)
    ) {
        Image(
            /*...*/
            modifier = Modifier.requiredSize(150.dp)
        )
        Column { /*...*/ }
    }
}
```

ModifierSnippets.kt



```
@Composable
fun ArtistCard(artist: Artist) {
    Row(/*...*/) {
        Column {
            Text(artist.name)
            Text(
                text = artist.lastSeenOnline,
                modifier = Modifier.offset(x = 4.dp)
            )
        }
    }
}
```

ModifierSnippets.kt

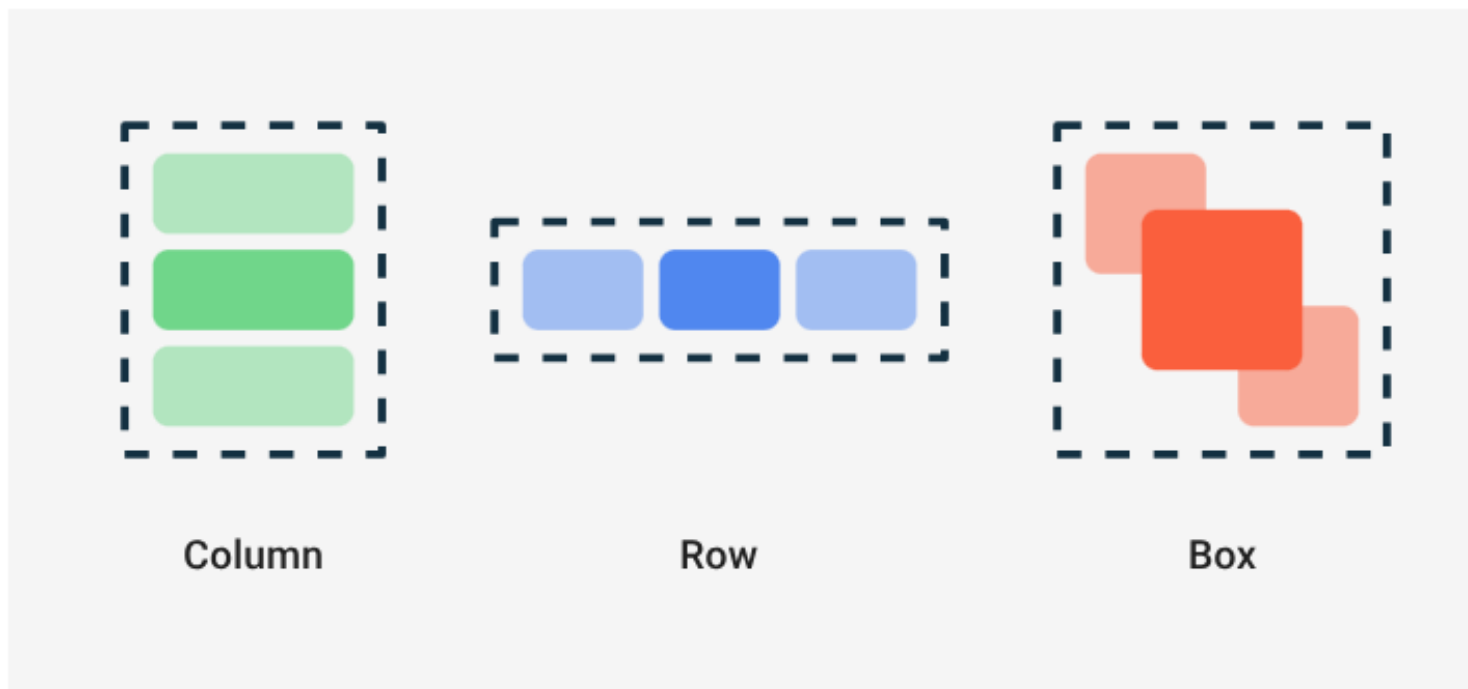




Layout

- Jetpack Compose makes it easy to design an efficient layout for your app.

There are three main layouts:



Layout

```
@Composable
fun ArtistCardColumn() {
    Column {
        Text("Alfred Sisley")
        Text("3 minutes ago")
    }
}
```

Alfred Sisley

3 minutes ago

```
@Composable
fun ArtistCardRow(artist: Artist) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Image(bitmap = artist.image, contentDescription = "Artist image")
        Column {
            Text(artist.name)
            Text(artist.lastSeenOnline)
        }
    }
}
```



Alfred Sisley

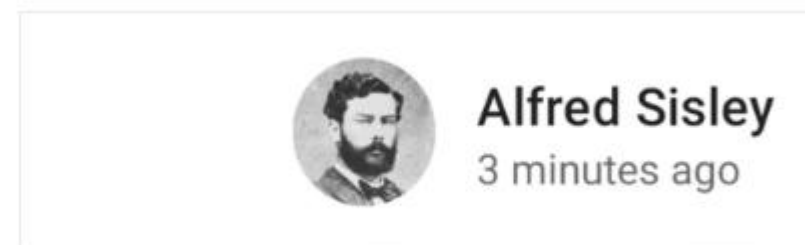
3 minutes ago

```
@Composable
fun ArtistAvatar(artist: Artist) {
    Box {
        Image(bitmap = artist.image, contentDescription = "Artist image")
        Icon(Icons.Filled.Check, contentDescription = "Check mark")
    }
}
```



- To set children's position within a **Row**, set the **horizontalArrangement** and **verticalAlignment** arguments. For a **Column**, set the **verticalArrangement** and **horizontalAlignment** arguments:

```
@Composable
fun ArtistCardArrangement(artist: Artist) {
    Row(
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.End
    ) {
        Image(bitmap = artist.image, contentDescription = "Artist image")
        Column { /*...*/ }
    }
}
```



Layout

- In compose. Parents measure before their children, but are sized and placed after the children.

```
@Composable
fun SearchResult() {
    Row {
        Image(
            // ...
        )
        Column {
            Text(
                // ...
            )
            Text(
                // ...
            )
        }
    }
}
```

```
SearchResult
  Row
    Image
    Column
      Text
      Text
```

In the beside sample, the UI tree layout follows this order:

1. The root node **Row** is asked to measure.
2. The root node **Row** asks its first child, **Image**, to measure.
3. **Image** is a leaf node (that is, it has no children), so it reports a size and returns placement instructions.
4. The root node **Row** asks its second child, **Column**, to measure.
5. The **Column** node asks its first **Text** child to measure.
6. The first **Text** node is a leaf node, so it reports a size and returns placement instructions.
7. The **Column** node asks its second **Text** child to measure.
8. The second **Text** node is a leaf node, so it reports a size and returns placement instructions.
9. Now that the **Column** node has measured, size, placed its children, it can determine its own size and placement.
10. Now that the root node **Row** has measured, sized, and placed its children, it can determine its own size and placement.

Layout



```
@Composable
fun SearchResult(...) {
    Row(...) {
        Image(...)
        Column(...) {
            Text(...)
            Text(...)
        }
    }
}
```

SearchResult

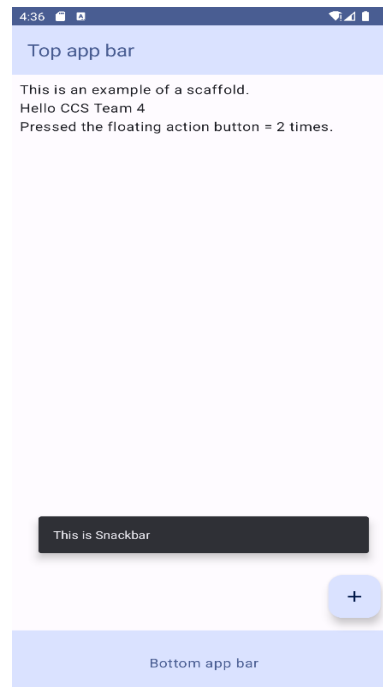
Row	1 measure	10 size	place
Image	2 measure	3 size	place
Column	4 measure	9 size	place
Text	5 measure	6 size	place
Text	7 measure	8 size	place

Scaffold

A scaffold is a fundamental structure that provides a standardized platform for complex user interfaces. It holds together different parts of the UI, such as app bars and floating action buttons, giving apps a coherent look and feel.

Components in a scaffold:

- Top Bar
- Bottom Bar
- Snack Bar Host
- Floating Action Button
- Content



```
@Composable
fun ScaffoldExample() {
    var presses by remember { mutableIntStateOf(0) }

    Scaffold(
        topBar = {
            TopAppBar(
                colors = topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer,
                    titleContentColor = MaterialTheme.colorScheme.primary,
                ),
                title = {
                    Text("Top app bar")
                }
            )
        },
        bottomBar = {
            BottomAppBar(
                containerColor = MaterialTheme.colorScheme.primaryContainer,
                contentColor = MaterialTheme.colorScheme.primary,
            ) {
                Text(
                    modifier = Modifier
                        .fillMaxWidth(),
                    textAlign = TextAlign.Center,
                    text = "Bottom app bar",
                )
            }
        },
        floatingActionButton = {
            FloatingActionButton(onClick = { presses++ }) {
                Icon(Icons.Default.Add, contentDescription = "Add")
            }
        },
        { innerPadding ->
            Column(
                modifier = Modifier
                    .padding(innerPadding),
                verticalArrangement = Arrangement.spacedBy(16.dp),
            ) {
                Text(
                    modifier = Modifier.padding(8.dp),
                    text =
                        """
                            This is an example of a scaffold. It uses the Scaffold composable's parameters

                            It also contains some basic inner content, such as this text.

                            You have pressed the floating action button $presses times.
                        """.trimIndent(),
                )
            }
        }
    )
}
```

Scaffold.kt

- Surface is a basic building block for displaying content and can be used to wrap other composable to provide a background color, elevation, padding, and other layout properties.
- Parameters of a surface:
 - ✓ **modifier**: Modifier to apply to the surface
 - ✓ **color**: Background color of the surface
 - ✓ **elevation**: Elevation of the surface, which adds a shadow effect
 - ✓ **shape**: Shapes of the surface, such as a rounded rectangle or a cut-corner rectangle
 - ✓ **border**: Border around the surface
 - ✓ **contentColor**: Foreground color of the surface's content
 - ✓ **onClick**: Click listener for the surface
 - ✓ **enabled**: Whether the surface is enabled or disabled
 - ✓ **interactionSource**: Interaction source for the surface
 - ✓ **indication**: Indication to show when the surface interacted with
 - ✓ **semantics**: Accessibility properties for the surface
 - ✓ **content**: The content of the surface, specified as a composable function

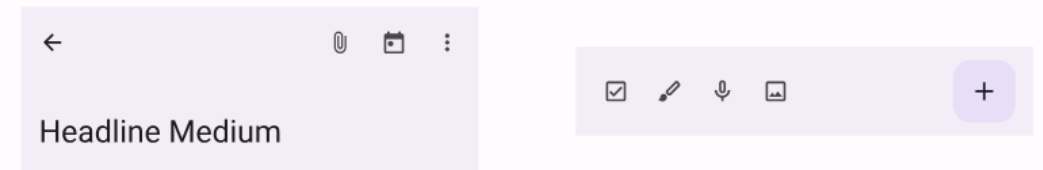
Refer: [Surface in jetpack compose](#)

App bars are containers that provide the user access to key features and navigation items. There are two types of app bars, top app bars and bottom app bars. Their respective appearance and purpose are as follows:

API Surface:

- **Title:** The text that appears across the app bar.
- **NavigationIcon:** The primary icon for navigation. Appears on the left of the app bar.
- **Actions:** Icons that provide the user access to key actions. They appear on the right of the app bar.
- **ScrollBehavior:** Determines how the top app bar responds to scrolling of the scaffold's inner content.
- **Colors:** Determines how the app bar appears.

Type	Appearance	Purpose
Top app bar	Across the top of the screen.	Provides access to key tasks and information. Generally hosts a title, core action items, and certain navigation items.
Bottom app bar	Across the bottom of the screen.	Typically includes core navigation items. May also provide access to other key actions, such as through a contained floating action button.



AppBar

Here are examples for top bar and bottom bar

```
Scaffold(  
  topBar = {  
    CenterAlignedTopAppBar(  
      colors = TopAppBarDefaults.topAppBarColors(  
        containerColor = MaterialTheme.colorScheme.primaryContainer,  
        titleContentColor = MaterialTheme.colorScheme.primary,  
      ),  
      title = {  
        Text(  
          text: "Centered Top App Bar",  
          maxLines = 1,  
          overflow = TextOverflow.Ellipsis  
        )  
      },  
      navigationIcon = {  
        IconButton(onClick = { /* do something */ }) {  
          Icon(  
            imageVector = Icons.Filled.ArrowBack,  
            contentDescription = "Localized description"  
          )  
        }  
      },  
      actions = { this: RowScope  
        IconButton(onClick = { /* do something */ }) {  
          Icon(  
            imageVector = Icons.Filled.Menu,  
            contentDescription = "Localized description"  
          )  
        }  
      },  
      scrollBehavior = scrollBehavior,  
    )  
  },  
)
```

```
bottomBar = {  
  BottomAppBar(  
    actions = { this: RowScope  
      IconButton(onClick = { }) {  
        Icon(Icons.Filled.Check, contentDescription = "Localized description")  
      }  
      IconButton(onClick = { }) {  
        Icon(  
          Icons.Filled.Edit,  
          contentDescription = "Localized description",  
        )  
      }  
      IconButton(onClick = { }) {  
        Icon(  
          Icons.Filled.Mic,  
          contentDescription = "Localized description",  
        )  
      }  
      IconButton(onClick = { }) {  
        Icon(  
          Icons.Filled.Image,  
          contentDescription = "Localized description",  
        )  
      }  
    },  
    floatingActionButton = {  
      FloatingActionButton(  
        onClick = { /* do something */ },  
        containerColor = BottomAppBarDefaults.bottomAppBarFabColor,  
        elevation = FloatingActionButtonDefaults.bottomAppBarFabElevation()  
      ) {  
        Icon(Icons.Filled.Add, contentDescription: "Localized description")  
      }  
    }  
  )  
}
```



Navigation Drawer

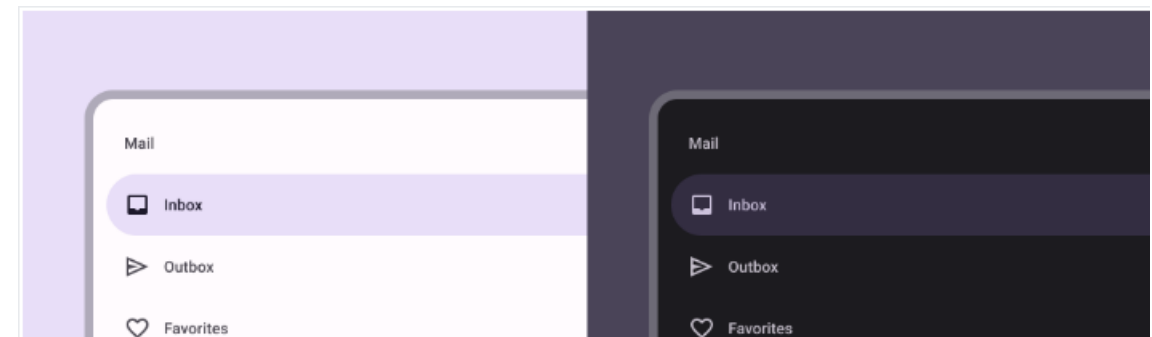
The navigation drawer component is a slide-in menu that lets users navigate to various sections of your app. Users can activate it by swiping from the side or tapping a menu icon

Consider these three use cases for implementing a Navigation Drawer

- **Content organization:** Enable users to switch between different categories, such as in news or blogging apps
- **Account management:** Provide quick links to account settings and profile sections in apps with user accounts
- **Feature discovery:** Organize multiple features and settings in a single menu to facilitate user discovery and access in complex apps

```
val drawerState = rememberDrawerState(initialValue = DrawerValue.Closed)
val scope = rememberCoroutineScope()
ModalNavigationDrawer(
    drawerState = drawerState,
    drawerContent = {
        ModalDrawerSheet { /* Drawer content */ }
    },
) {
    Scaffold(
        floatingActionButton = {
            ExtendedFloatingActionButton(
                text = { Text("Show drawer") },
                icon = { Icon(Icons.Filled.Add, contentDescription = "") },
                onClick = {
                    scope.launch {
                        drawerState.apply {
                            if (isClosed) open() else close()
                        }
                    }
                }
            )
        }
    ) { contentPadding ->
        // Screen content
    }
}
```

MaterialLayoutSnippets.kt



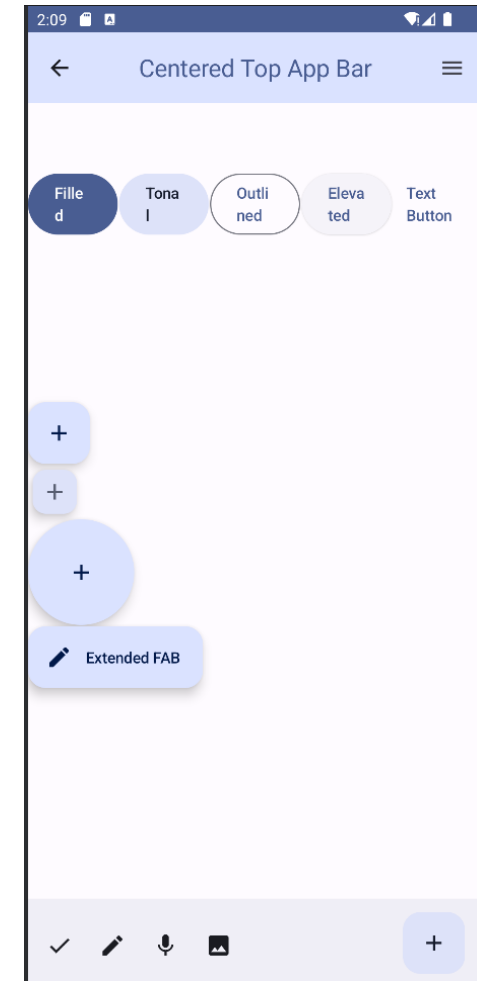
Floating Action Button

A Floating Action Button (FAB) is a high-emphasis button that lets the user perform a primary action in an application, anchored to the bottom right of the screen.

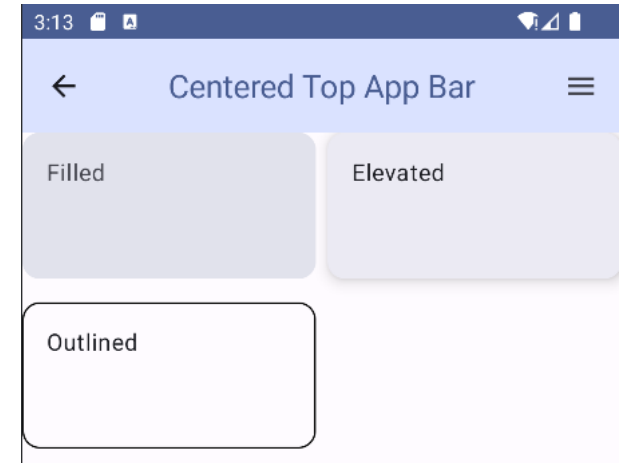
In Material Design, there are four types of FAB:

- ✓ **FAB:** A floating action button of ordinary size.
- ✓ **Small FAB:** A smaller floating action button.
- ✓ **Large FAB:** A larger floating action button.
- ✓ **Extended FAB:** A floating action button that contains more than just an icon.

```
Column(  
  modifier = Modifier  
    .padding(innerPadding),  
  verticalArrangement = Arrangement.spacedBy(1.dp),  
{ this: ColumnScope  
  FloatingActionButton(  
    onClick = { },  
  ) {  
    Icon(Icons.Filled.Add, contentDescription: "Floating action button.")  
  }  
  SmallFloatingActionButton(  
    onClick = { },  
    containerColor = MaterialTheme.colorScheme.secondaryContainer,  
    contentColor = MaterialTheme.colorScheme.secondary  
  ) {  
    Icon(Icons.Filled.Add, contentDescription: "Small floating action button.")  
  }  
  LargeFloatingActionButton(  
    onClick = { },  
    shape = CircleShape,  
  ) {  
    Icon(Icons.Filled.Add, contentDescription: "Large floating action button")  
  }  
  ExtendedFloatingActionButton(  
    onClick = { },  
    icon = { Icon(Icons.Filled.Edit, contentDescription: "Extended floating action button.") },  
    text = { Text(text = "Extended FAB") },  
  )  
}
```



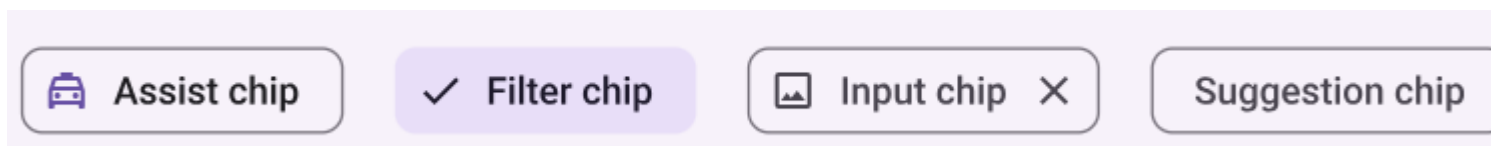
- The `Card` composable acts as a Material Design container for your UI. Cards typically present a single coherent piece of content.
- Card is generally a smaller UI element inside a larger layout
- Some key parameters to note are the following
 - **elevation**: Adds a shadow to the component that makes it appear elevated above the background.
 - **colors**: Uses the Card Colors type to set the default color of both the container and any children.
 - **enabled**: If you pass false for this parameter, the card appears as disabled and does not respond to user input.
 - **onClick**: Ordinarily, a Card does not accept click events.
- Types of card:
 - Filled card
 - Elevated Card
 - Outlined Card



```
Column(
    modifier = Modifier
        .weight(1f)
        .fillMaxHeight(),
    verticalArrangement = Arrangement.spacedBy(10.dp)
) {
    this: ColumnScope
    Card(
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant,
        ),
        modifier = Modifier
            .size(width = 240.dp, height = 100.dp)
    ) {
        this: ColumnScope
        Text(
            text = "Filled",
            modifier = Modifier
                .padding(10.dp),
            textAlign = TextAlign.Center,
        )
    }
    OutlinedCard(
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface,
        ),
        border = BorderStroke(1.dp, Color.Black),
        modifier = Modifier
            .size(width = 240.dp, height = 100.dp)
    ) {
        this: ColumnScope
        Text(
            text = "Outlined",
            modifier = Modifier
                .padding(10.dp),
            textAlign = TextAlign.Center,
        )
    }
}
```

```
Column(
    modifier = Modifier
        .weight(1f)
        .fillMaxHeight()
) {
    this: ColumnScope
    ElevatedCard(
        elevation = CardDefaults.cardElevation(
            defaultElevation = 6.dp
        ),
        modifier = Modifier
            .size(width = 240.dp, height = 100.dp)
    ) {
        this: ColumnScope
        Text(
            text = "Elevated",
            modifier = Modifier
                .padding(10.dp),
            textAlign = TextAlign.Center,
        )
    }
}
```

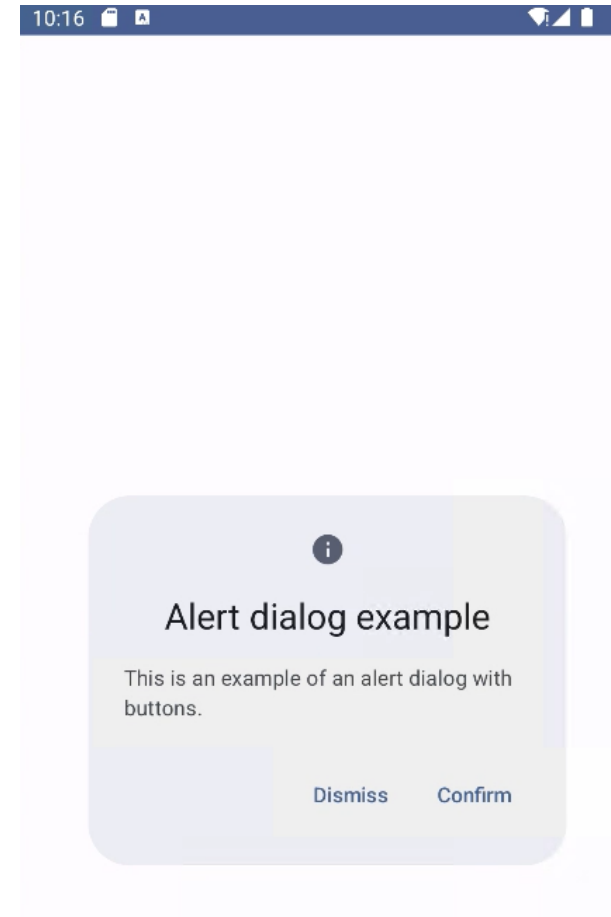
- The **Chip** component is a compact interactive UI element. It represents complex entities like a contact or tag, often with an icon and label. It can be checkable, dismissible, or clickable.
- Types of chip
 - **Assist:** Guides the user during a task. Often appears as a temporary UI element in response to user input.
 - **Filter:** Allows users to refine content from a set of options. They can be selected or unselected, and may include a checkmark icon when selected.
 - **Input:** Represents user-provided information, such as selections in a menu. They can contain an icon and text, and provide an 'X' for removal.
 - **Suggestion:** Provides recommendations to the user based on their recent activity or input. Typically appear beneath an input field to prompt user actions.



Dialog

- The Dialog component displays pop up messages or requests user input on a layer above the main app content. It creates an interruptive UI experience to capture user attention.

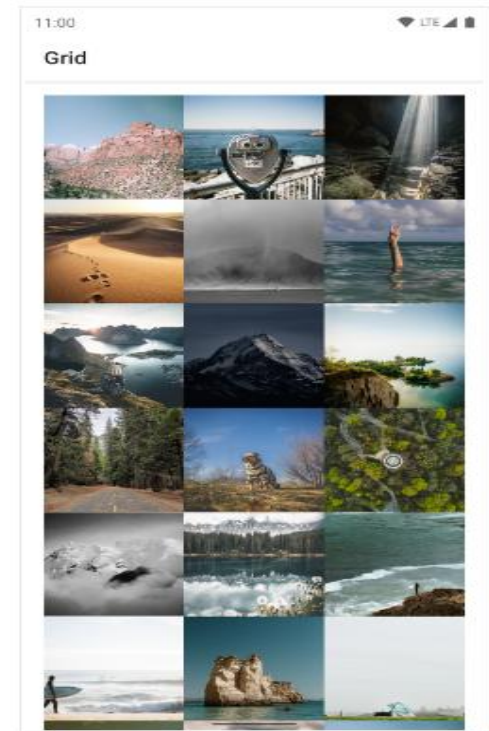
```
AlertDialog(  
  icon = {  
    Icon(icon, contentDescription = "Example Icon")  
  },  
  title = {  
    Text(text = dialogTitle)  
  },  
  text = {  
    Text(text = dialogText)  
  },  
  onDismissRequest = {  
    onDismissRequest()  
  },  
  confirmButton = {  
    TextButton(  
      onClick = {  
        onConfirmation()  
      }  
    ) { this: RowScope  
      Text( text: "Confirm")  
    }  
  },  
  dismissButton = {  
    TextButton(  
      onClick = {  
        onDismissRequest()  
      }  
    ) { this: RowScope  
      Text( text: "Dismiss")  
    }  
  }  
)
```



List and Grid

- We can use Column and Row to display a numbers of items on screen same with ListView, but for a large items, Row and Column can causes performance issue.
- In Compose, we can use bellow composables to optimize data loading and smooth scrolling.
 - LazyColumn and LazyRow: Display items same with ListView
 - LazyVerticalGrid, LazyHorizontalGrid: Display items same with GridView

Refer: [Lists and Grids](#)



LazyColumn

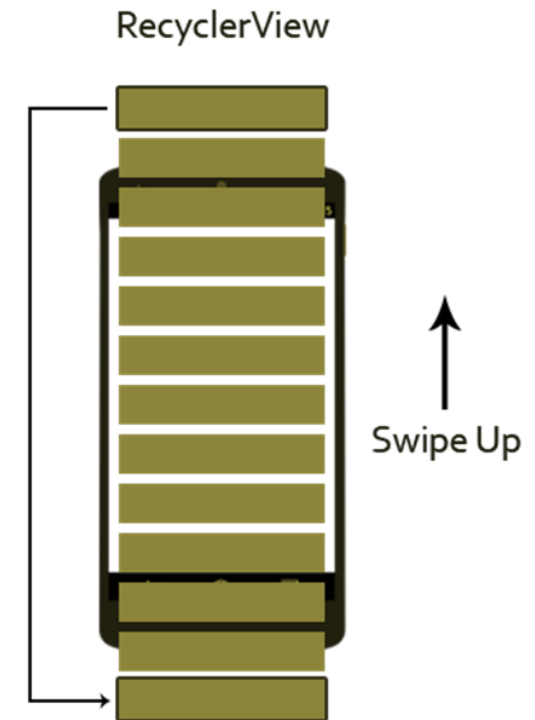
A LazyColumn is a vertically scrolling list that only composes and lays out the currently visible items. It's similar to a RecyclerView in the classic Android View system.

```
@Composable
fun LazyColumnExample() {

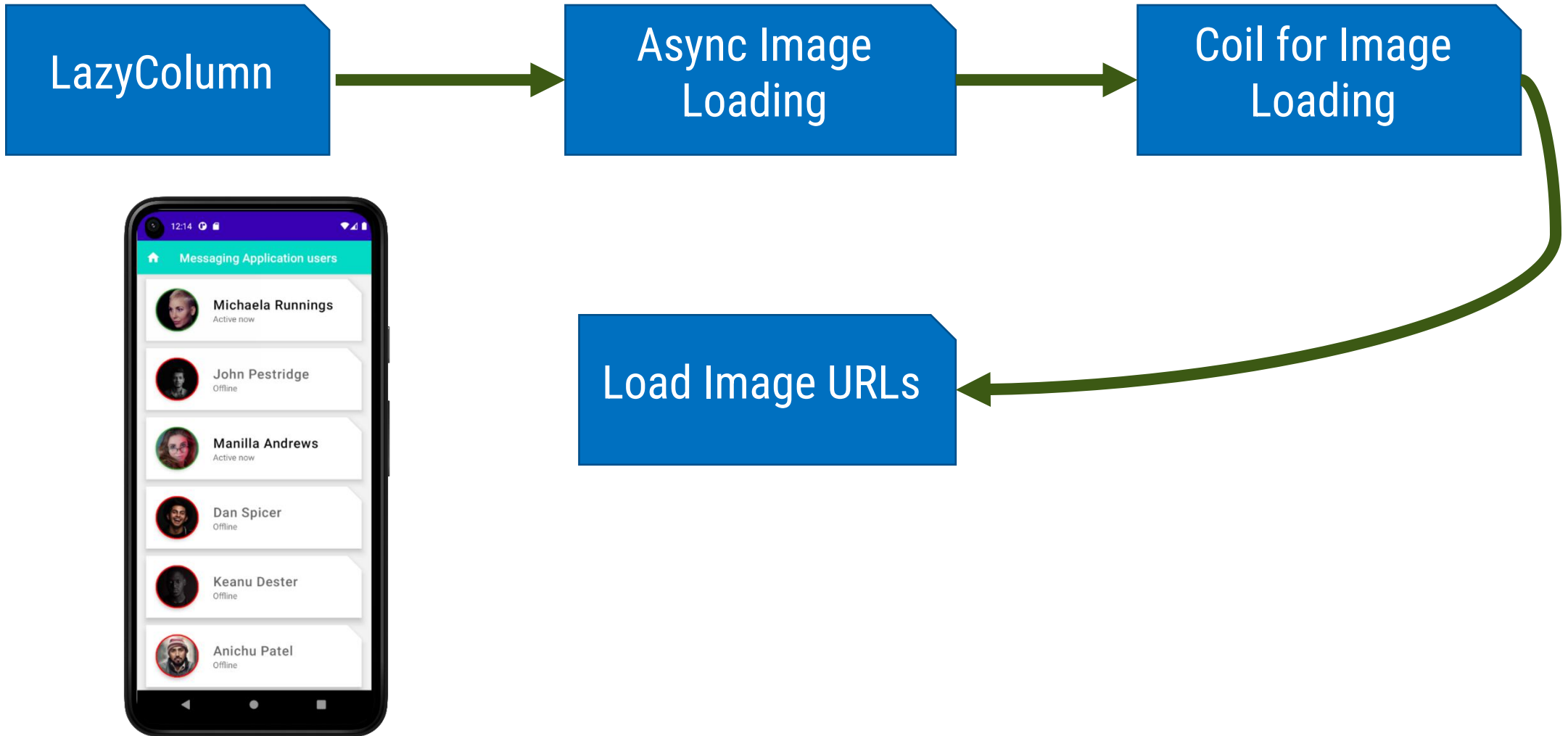
    // Person list
    val dataList = listOf("Abhishek", "Harshit", "Gaurav", "Avin", "Avish", "Tanu", ...)

    LazyColumn {
        items(dataList) { item ->
            // This lambda will be called for each item in the dataList
            // and create a composable UI element for that item.
            ListItem(text = item)
        }
    }
}

@Composable
fun ListItem(text: String) {
    // A composable representing a single item in the list
    // Customize it to display the data as per your requirement.
    // For simplicity, we'll just display the text here.
    Text(
        text,
        fontSize = 30.sp,
        fontWeight = FontWeight.SemiBold,
        fontStyle = FontStyle.Normal,
        modifier = Modifier.padding(vertical = 20.dp, horizontal = 30.dp)
    )
}
```



LazyColumn



LazyVerticalGrid

The LazyVerticalGrid and LazyHorizontalGrid composables provide support for displaying items in a grid. A Lazy vertical grid will display its items in a vertically scrollable container, spanned across multiple columns, while the Lazy horizontal grids will have the same behaviour on the horizontal axis

```
LazyVerticalGrid(  
    columns = GridCells.Adaptive(minSize = 128.dp)  
) {  
    items(photos) { photo ->  
        PhotoItem(photo)  
    }  
}
```

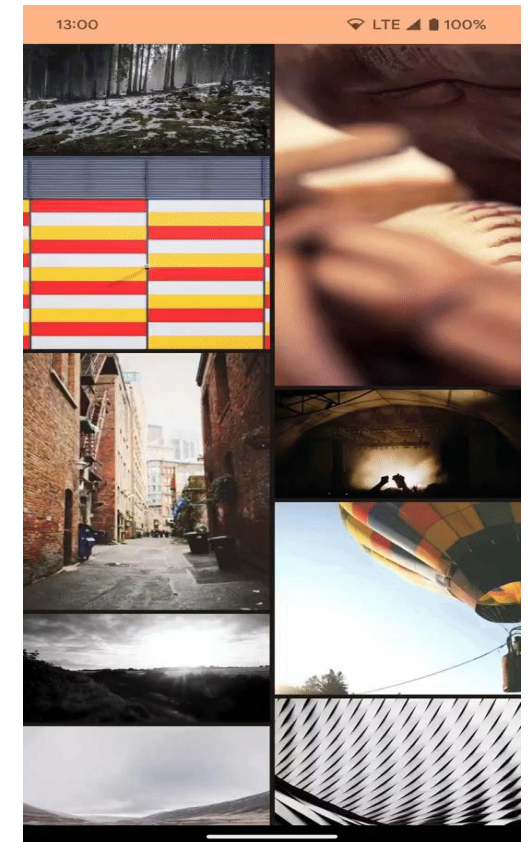


Lazy Staggered Grid

LazyVerticalStaggeredGrid and LazyHorizontalStaggeredGrid are composables that allow you to create a lazy-loaded, staggered grid of items.

```
LazyVerticalStaggeredGrid(  
    columns = StaggeredGridCells.Adaptive(200.dp),  
    verticalItemSpacing = 4.dp,  
    horizontalArrangement = Arrangement.spacedBy(4.dp),  
    content = {  
        items(randomizedPhotos) { photo ->  
            AsyncImage(  
                model = photo,  
                contentScale = ContentScale.Crop,  
                contentDescription = null,  
                modifier = Modifier.fillMaxWidth().wrapContentHeight()  
            )  
        }  
    },  
    modifier = Modifier.fillMaxSize()  
)
```

LazyListSnippets.kt



Pager

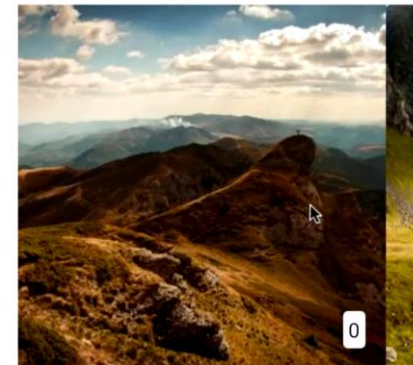
Pager in compose is similar with ViewPager in traditional android development. To flip through content in a left and right or up and down manner. We can use **HorizontalPager** or **VerticalPager**.

By default, **HorizontalPager** takes up the full width of screen, **VerticalPager** takes up the full height

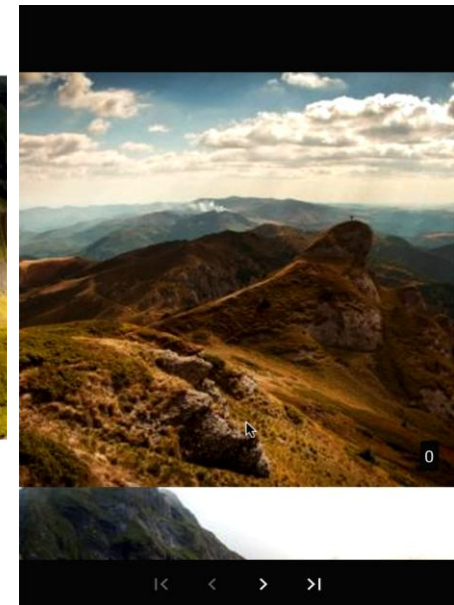
- Pager uses lazy creation. As the user scrolls the pages, the composable removes any pages which are no longer required.
- By default, the pager only loads the visible pages on-screen. To load more pages offscreen, set **beyondBoundsPageCount** to a value higher than zero

```
// Display 10 items
val pagerState = rememberPagerState(pageCount = {
    10
})
HorizontalPager(state = pagerState) { page ->
    // Our page content
    Text(
        text = "Page: $page",
        modifier = Modifier.fillMaxWidth()
    )
}
```

```
// Display 10 items
val pagerState = rememberPagerState(pageCount = {
    10
})
VerticalPager(state = pagerState) { page ->
    // Our page content
    Text(
        text = "Page: $page",
        modifier = Modifier.fillMaxWidth()
    )
}
```



|< < > >|



|< < > >|

Refer: [Pager](#)

FlowLayout

FlowRow and **FlowColumn** are composable that are similar to **Row** and **Column**, but differ in that items flow in to the next line when the container runs out of space. We can control the number of line by setting **maxItemsInEachRow** and **maxItemsInEachColumn**.

```
@Composable
private fun FlowRowSimpleUsageExample() {
    FlowRow(modifier = Modifier.padding(8.dp)) {
        ChipItem("Price: High to Low")
        ChipItem("Avg rating: 4+")
        ChipItem("Free breakfast")
        ChipItem("Free cancellation")
        ChipItem("£50 pn")
    }
}
```

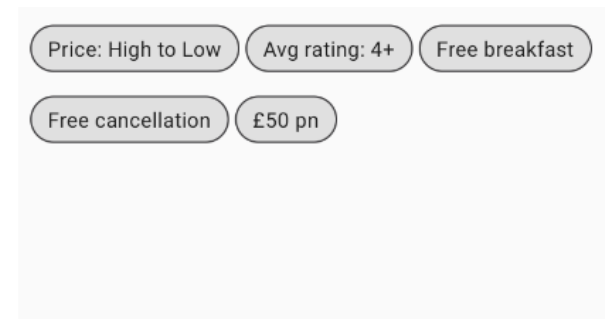
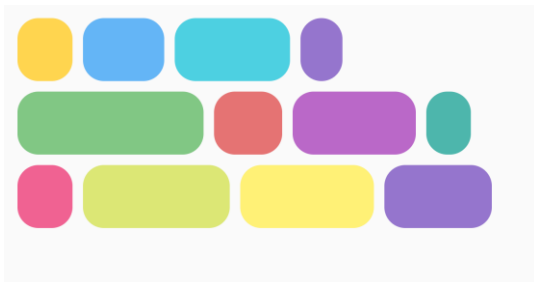
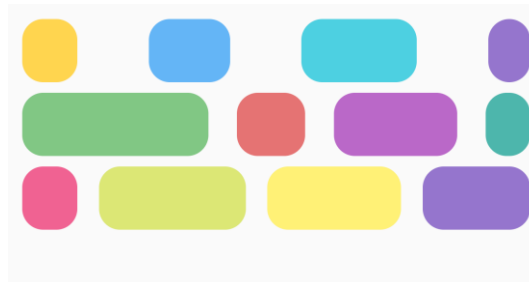


Figure 1. Example of FlowRow



Arrangement.Start (Default)



Arrangement.SpaceBetween

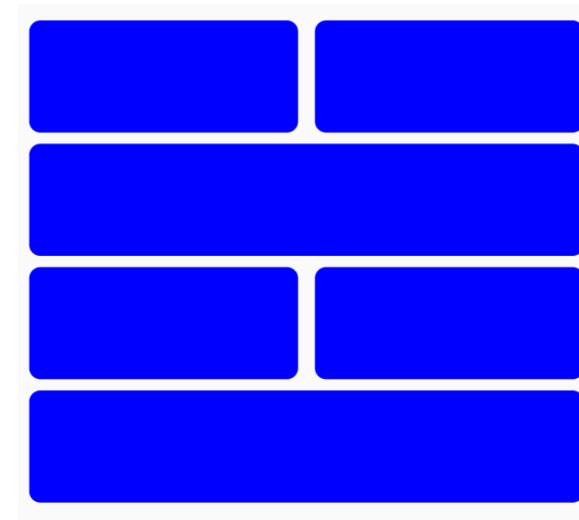


Arrangement.Center

Custom Flow Layout example

Here is an example for custom flow layout

```
FlowLayout(  
    modifier = Modifier.padding(4.dp),  
    horizontalArrangement = Arrangement.spacedBy(4.dp),  
    maxItemsInEachRow = 2  
) {  
    val itemModifier = Modifier  
        .padding(4.dp)  
        .height(80.dp)  
        .clip(RoundedCornerShape(8.dp))  
        .background(Color.Blue)  
    repeat(6) { item ->  
        // if the item is the third item, don't use weight modifier, but rather fillMaxWidth  
        if ((item + 1) % 3 == 0) {  
            Spacer(modifier = itemModifier.fillMaxWidth())  
        } else {  
            Spacer(modifier = itemModifier.weight(0.5f))  
        }  
    }  
}
```



ConstraintsLayout

ConstraintLayout is a layout that allows you to place composables relative to other composables on the screen. It is an alternative to using multiple nested Row, Column, Box and other custom layout elements. ConstraintLayout is useful when implementing larger layouts with more complicated alignment requirement

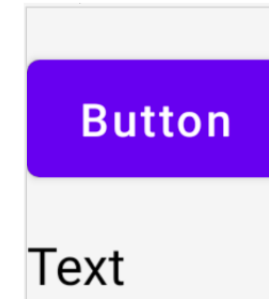
```
@Composable
fun ConstraintLayoutContent() {
    ConstraintLayout {
        // Create references for the composables to constrain
        val (button, text) = createRefs()

        Button(
            onClick = { /* Do something */ },
            // Assign reference "button" to the Button composable
            // and constrain it to the top of the ConstraintLayout
            modifier = Modifier.constrainAs(button) {
                top.linkTo(parent.top, margin = 16.dp)
            }
        ) {
            Text("Button")
        }

        // Assign reference "text" to the Text composable
        // and constrain it to the bottom of the Button composable
        Text(
            "Text",
            Modifier.constrainAs(text) {
                top.linkTo(button.bottom, margin = 16.dp)
            }
        )
    }
}
```

ConstraintLayoutSnippets.kt

This code constrains the top of the `Button` to the parent with a margin of `16.dp` and a `Text` to the bottom of the `Button` also with a margin of `16.dp`.



Refer: [Constraintlayout](#)

State in compose

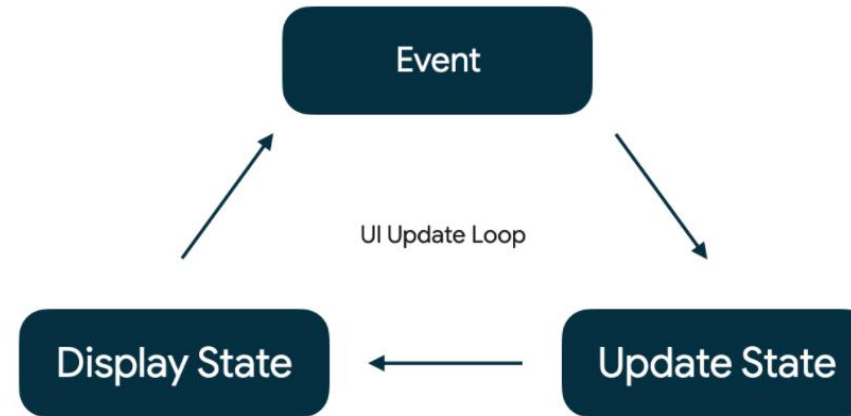
What is State

State in an app is **any value that can change** overtime, E.g.

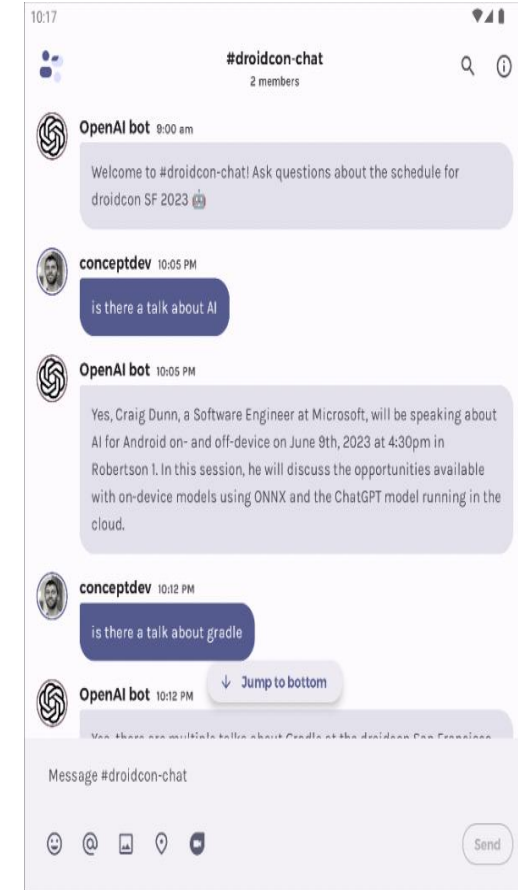
- The most recent messages received in a chat app
- The user's profile photo
- The scroll position in a list of items

Unidirectional data flow:

- **Event** - An event is generated by the user or another part of the program.
- **Update State** – An event handler changes the state that is used by the UI
- **Display State** – The UI is updated to display the new state

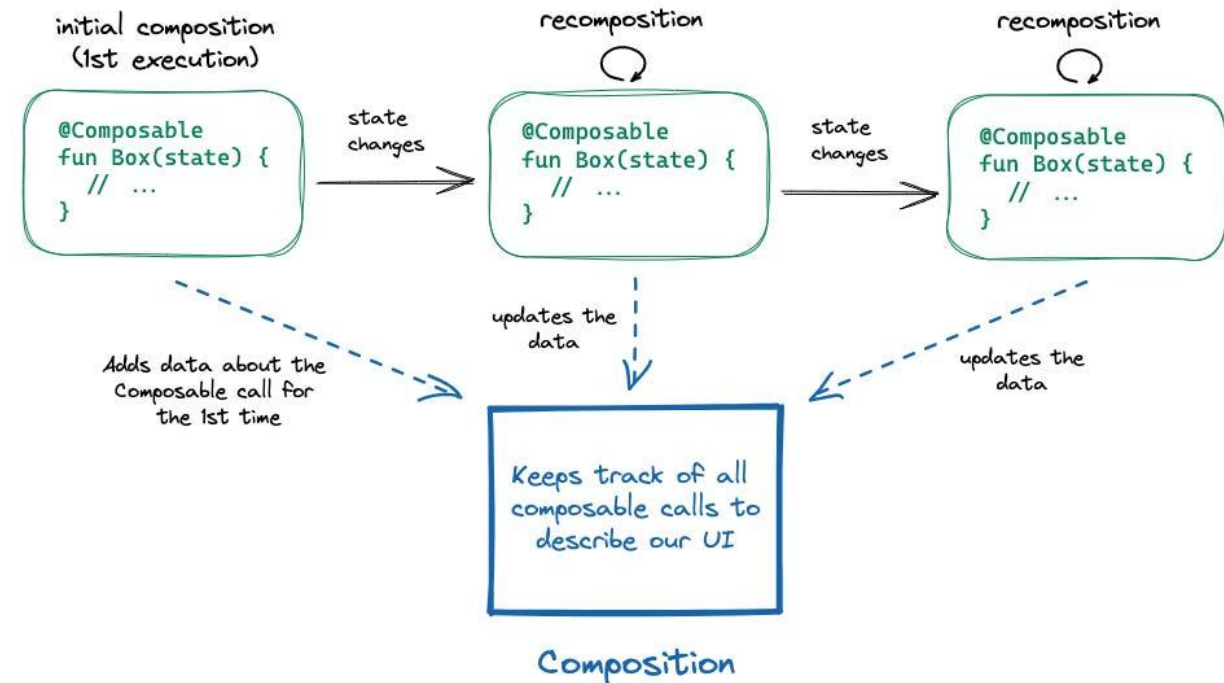


Application UI flow



State as part of Recomposition

- Compose is declarative → the only way to update by calling the same composable with new arguments (UI state)
- Any time a state is updated a *Recomposition* takes place
- Any action that causes the modification of state is called an “event”



State Example

- A composable is that uses `remember` to store an object creates internal state → Stateful composable
- A composable that doesn't hold any state → Stateless composable
- An easy way to achieve stateless is by using *state hoisting*!

Refer: [State](#)

```
@Composable
fun MainScreen() {
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.SpaceEvenly,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        GreetingList()
    }
}

@Composable
fun GreetingList() {
    val greetingListState = remember { mutableStateListOf<String>("John", "Amanda") }
    for(name in greetingListState) {
        Greeting(name = name)
    }

    Button(onClick = { greetingListState.add("Michael") }) {
        Text("Add new name")
    }
}

@Composable
fun Greeting(name: String) {
    Text(
        text = "Hello $name!",
        style = MaterialTheme.typography.h5
    )
}
```

Code: [MainActivity.kt](#)

State Hoisting(Lifting up state)

State hoisting is a pattern of moving state to a composable's caller to make a composable stateless.

Pattern:

- **value: T**: the current value to display
- **onValueChange: (T) -> Unit**: an event that requests the value to change, where T is the proposed new value

Advantages

- **Single source of truth**: By moving state instead of duplicating it, we're ensuring there's only one source of truth. This helps avoid bugs.
- **Encapsulated**: Only stateful composables can modify their state. It's completely internal.
- **Shareable**: Hoisted state can be shared with multiple composables.
- **Interceptable**: callers to the stateless composables can decide to ignore or modify events before changing the state.
- **Decoupled**: the state for the stateless composables may be stored anywhere. For example, it's now possible to move name into a ViewModel.

Refer: [State-hoisting](#) and [Jetpack-compose-state](#)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MainScreen()
        }
    }
}

@Composable
fun MainScreen() {
    val greetingListState = remember {
        mutableStateListOf<String>("John", "Amanda")
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.SpaceEvenly,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        GreetingList(greetingListState) {
            greetingListState.add("Michael")
        }
    }
}

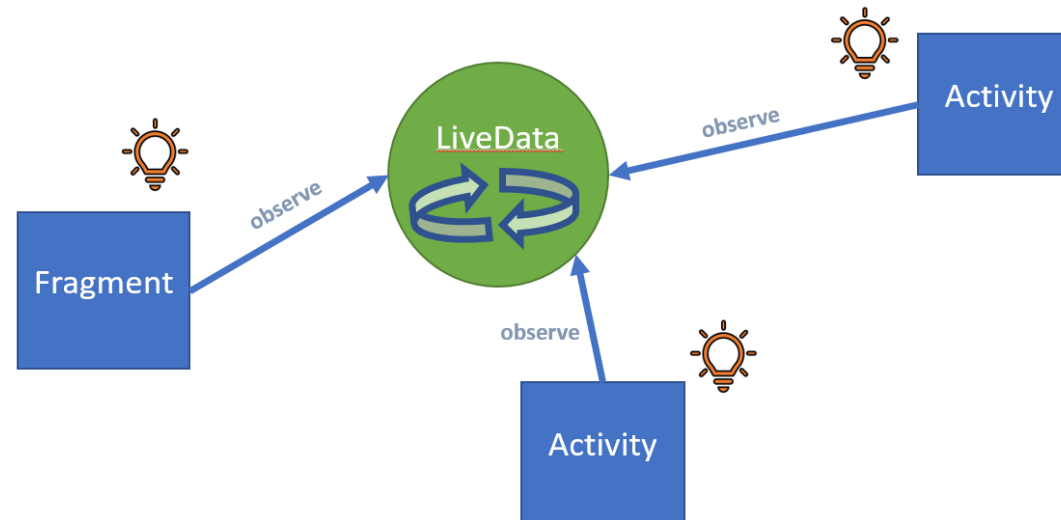
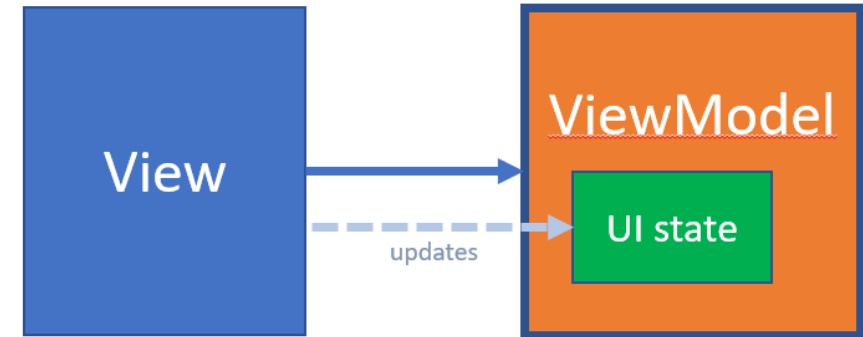
@Composable
fun GreetingList(namesList: List<String>, buttonClick: () -> Unit) {
    for(name in namesList) {
        Greeting(name = name)
    }

    Button(onClick = buttonClick) {
        Text("Add new name")
    }
}

@Composable
fun Greeting(name: String) {
    Text(
        text = "Hello $name!",
        style = MaterialTheme.typography.h5
    )
}
```

ViewModel and LiveData

- **ViewModel** is a class that is responsible for preparing and managing the state of the UI for Fragments or Activities.
- LiveData is a lifecycle-aware observable data holder



Lift up state to ViewModel

ViewModels suitable for

- Providing access to the **business logic**
- Preparing the application data for presentation on the screen

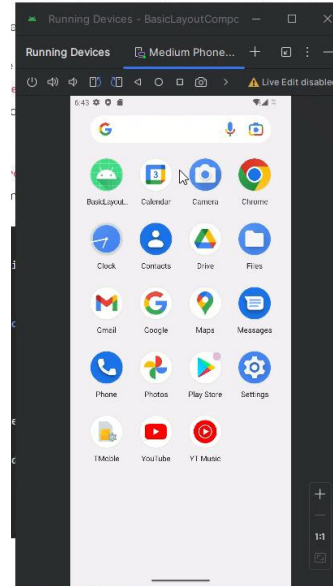
```
class MainViewModel: ViewModel() {  
    val textFieldState = MutableLiveData("")  
  
    fun onTextChanged(newText: String) {  
        textFieldState.value = newText  
    }  
}
```

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            MainScreen()  
        }  
    }  
}  
  
@Composable  
fun MainScreen(viewModel: MainViewModel = MainViewModel()) {  
    val newNameStateContent = viewModel.textFieldState.observeAsState("")  
  
    Column(  
        modifier = Modifier.fillMaxSize(),  
        verticalArrangement = Arrangement.SpaceEvenly,  
        horizontalAlignment = Alignment.CenterHorizontally  
    ) {  
        GreetingMessage(  
            newNameStateContent.value  
        ) { newName -> viewModel.onTextChanged(newName) }  
    }  
}  
  
@Composable  
fun GreetingMessage(  
    textFieldValue: String,  
    textFieldUpdate: (newName: String) -> Unit  
) {  
    TextField(value = textFieldValue, onValueChange = textFieldUpdate)  
    Button(onClick = { }) {  
        Text(textFieldValue)  
    }  
}  
  
@Composable  
fun Greeting(name: String) {  
    Text(  
        text = "Hello $name!",  
        style = MaterialTheme.typography.h5  
    )  
}
```

Side Effect

- A **side-effect** is a change to the state of the app that happens outside the scope of a composable function
- Due to the lifecycle of composables and properties such as unpredictable recompositions, executing recompositions of composables in different orders, or recompositions that can be discarded

Composables should ideally be **side-effect free**.



```
38
39 @Composable
40 fun WithoutSideEffectExample() {
41     var counter by remember { mutableIntStateOf(0) }
42     val context = LocalContext.current
43
44     // on every recomposition, this toast will show
45     Toast.makeText(context, text = "Hello", Toast.LENGTH_LONG).show()
46
47     Column(
48         modifier = Modifier.fillMaxSize(),
49         verticalArrangement = Arrangement.Center,
50         horizontalAlignment = Alignment.CenterHorizontally
51     ) {
52         Text(
53             text = "Counter: $counter",
54             modifier = Modifier.padding(bottom = 16.dp)
55         )
56         Button(
57             onClick = { counter++ },
58             { this: RowScope
59                 Text(text = "Increment")
60             }
61         )
62     }
63 }
```

Effect Apis

State and effect use cases?

❑ Effect-Handlers

Suspended effect handler:

- `LaunchEffect`
- `RememberCoroutineScope`

Non-Suspended effect handler:

- `DisposableEffect`
- `SideEffect`

❑ Side-Effect States

- `RememberUpdatedState`
- `ProduceState`
- `DerivedStateOf`
- `SnapShotFlow`



Refer: [Side-effects](#)

Effect Handlers

LaunchedEffect: run suspend functions in the scope of a composable

- LaunchedEffect is a composable function that is used to launch a coroutine inside the scope of composable
- When LaunchedEffect enters the composition, it launches a coroutine and cancels when it leaves composition
- LaunchedEffect takes multiple keys as params and if any of the key changes it cancels the existing coroutine and launches again

```
40 @Composable
41 fun WithoutSideEffectExample() {
42     var counter by remember { mutableIntStateOf(0) }
43     val context = LocalContext.current
44
45     LaunchedEffect(key1 = true) { this: CoroutineScope
46         Toast.makeText(context, text = "Hello", Toast.LENGTH_LONG).show()
47     }
48
49     Column(
50         modifier = Modifier.fillMaxSize(),
51         verticalArrangement = Arrangement.Center,
52         horizontalAlignment = Alignment.CenterHorizontally
53     ) { this: ColumnScope
54         Text(
55             text = "Counter: $counter",
56             modifier = Modifier.padding(bottom = 16.dp)
57         )
58         Button(
59             onClick = { counter++ },
60             { this: RowScope
61                 Text(text = "Increment")
62             }
63         )
64     }
65 }
```

```
40 @Composable
41 fun WithoutSideEffectExample() {
42     var counter by remember { mutableIntStateOf(0) }
43     val context = LocalContext.current
44
45     LaunchedEffect(key1 = counter) { this: CoroutineScope
46         Toast.makeText(context, text = "Hello", Toast.LENGTH_LONG).show()
47     }
48
49     Column(
50         modifier = Modifier.fillMaxSize(),
51         verticalArrangement = Arrangement.Center,
52         horizontalAlignment = Alignment.CenterHorizontally
53     ) { this: ColumnScope
54         Text(
55             text = "Counter: $counter",
56             modifier = Modifier.padding(bottom = 16.dp)
57         )
58         Button(
59             onClick = { counter++ },
60             { this: RowScope
61                 Text(text = "Increment")
62             }
63         )
64     }
65 }
```

Effect Handlers

rememberCoroutineScope: obtain a composition-aware scope to launch coroutine outside a composable

- As **LaunchedEffect** is a composable function, it can only be used inside other composable

Using **rememberCoroutineScope** when you want to launch a coroutine outside of a composable, but scoped so that it will be automatically canceled once it leaves the composition

```
@Composable
fun SelfMessage() {
    val coroutineScope = rememberCoroutineScope()
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) { this: ColumnScope
        Button(
            onClick = {
                LaunchedEffect(key1 = true) { this: CoroutineScope
                    fetchData()
                }
            },
        ) { this: RowScope
            Text(text = "Fetch data")
        }
    }
}
```

```
67
68 @Composable
69 fun SelfMessage() {
70     val coroutineScope = rememberCoroutineScope()
71     Column(
72         modifier = Modifier.fillMaxSize(),
73         verticalArrangement = Arrangement.Center,
74         horizontalAlignment = Alignment.CenterHorizontally
75     ) { this: ColumnScope
76
77         Button(
78             onClick = {
79                 coroutineScope.launch { this: CoroutineScope
80                     val result = fetchData()
81                 }
82             },
83         ) { this: RowScope
84             Text(text = "Fetch data")
85         }
86     }
87 }
88
```


LaunchedEffect vs rememberCoroutineScope

- **LaunchedEffect** should be used when you want that some action must be taken when your composable is first launched/relaunched (or when the key parameter has changed). For example, when you want to request some data from your ViewModel or run some sort of animation...
- **rememberCoroutineScope** on the other hand, is specific to store the Coroutine scope allowing the code to launch some suspend function...
- The only relation between them is that you can also use a LaunchedEffect to launch a coroutine...

Effect Handlers

DisposableEffect: effects that require cleanup

- Use **DisposableEffect** for side effects that need to be cleaned up when the keys change or when the composable leaves the Composition
- If the **DisposableEffect** keys change, the composable needs to dispose (do the cleanup for) its current effect, and reset by calling the effect again
- In the code left, the effect will add the observer to the lifecycleOwner. If lifecycleOwner changes, the effect is disposed and restarted with the new lifecycleOwner.
- A DisposableEffect must include an onDispose clause as the final statement in its block of code. Otherwise, the IDE displays a build-time error.

```
@Composable
fun HomeScreen(
    lifecycleOwner: LifecycleOwner = LocalLifecycleOwner.current,
    onStart: () -> Unit, // Send the 'started' analytics event
    onStop: () -> Unit // Send the 'stopped' analytics event
) {
    // Safely update the current lambdas when a new one is provided
    val currentOnStart by rememberUpdatedState(onStart)
    val currentOnStop by rememberUpdatedState(onStop)

    // If 'lifecycleOwner' changes, dispose and reset the effect
    DisposableEffect(lifecycleOwner) {
        // Create an observer that triggers our remembered callbacks
        // for sending analytics events
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_START) {
                currentOnStart()
            } else if (event == Lifecycle.Event.ON_STOP) {
                currentOnStop()
            }
        }

        // Add the observer to the lifecycle
        lifecycleOwner.lifecycle.addObserver(observer)

        // When the effect leaves the Composition, remove the observer
        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }

    /* Home screen content */
}
```

SideEffectsSnippets.kt

Side-Effect States

SideEffect: publish Compose state to non-Compose code

- To share Compose state with objects not managed by Compose, use the SideEffect composable
- Using a **SideEffect** guarantees that the effect executes after every successful recomposition. On the other hand, it is incorrect to perform an effect before a successful recomposition is guaranteed, which is the case when writing the effect directly in a composable.

```
@Composable
fun rememberFirebaseAnalytics(user: User): FirebaseAnalytics {
    val analytics: FirebaseAnalytics = remember {
        FirebaseAnalytics()
    }

    // On every successful composition, update FirebaseAnalytics with
    // the userType from the current User, ensuring that future analytics
    // events have this metadata attached
    SideEffect {
        analytics.setUserProperty("userType", user.userType)
    }
    return analytics
}
```

SideEf

For example, your analytics library might allow you to segment your user population by attaching custom metadata ("user properties" in this example) to all subsequent analytics events. To communicate the user type of the current user to your analytics library, use SideEffect to update its value.

Side-Effect States

rememberUpdatedState: reference a value in an effect that shouldn't restart if the value changes

- **LaunchedEffect** restarts when one of the key parameters changes. However, in some situations you might want to capture a value in your effect that, if it changes, you do not want the effect to restart
- In order to do this, it is required to use **rememberUpdatedState** to create a reference to this value which can be captured and updated

```
@Composable
fun LandingScreen(onTimeout: () -> Unit) {

    // This will always refer to the latest onTimeout function that
    // LandingScreen was recomposed with
    val currentOnTimeout by rememberUpdatedState(onTimeout)

    // Create an effect that matches the lifecycle of LandingScreen.
    // If LandingScreen recomposes, the delay shouldn't start again.
    LaunchedEffect(true) {
        delay(SplashWaitTimeMillis)
        currentOnTimeout()
    }

    /* Landing screen content */
}
```

For example, suppose your app has a LandingScreen that disappears after some time. Even if LandingScreen is recomposed, the effect that waits for some time and notifies that the time passed shouldn't be restarted:

ProduceState

produceState: convert non-Compose state into Compose state

- **produceState** launches a coroutine scoped to the Composition that can push values into a returned State. Use it to convert non-Compose state into Compose state
- The **producer** is launched when **produceState** enters the Composition, and will be cancelled when it leaves the Composition. The returned **State conflates**; setting the same value won't trigger a recomposition.
- Even though **produceState** creates a coroutine, it can also be used to observe non-suspending sources of data. To remove the subscription to that source, use the **awaitDispose** function.

```
@Composable
fun loadNetworkImage(
    url: String,
    imageRepository: ImageRepository = ImageRepository()
): State<Result<Image>> {

    // Creates a State<T> with Result.Loading as initial value
    // If either `url` or `imageRepository` changes, the running producer
    // will cancel and will be re-launched with the new inputs.
    return produceState<Result<Image>>(initialValue = Result.Loading, url, imageRepository) {

        // In a coroutine, can make suspend calls
        val image = imageRepository.load(url)

        // Update State with either an Error or Success result.
        // This will trigger a recomposition where this State is read
        value = if (image == null) {
            Result.Error
        } else {
            Result.Success(image)
        }
    }
}
```

SideEffectsSnippets.kt

- The following example shows how to use **produceState** to load an image from the network.
- The loadNetworkImage composable function returns a State that can be used in other composables.

DerivedStateOf

DerivedStateOf: convert one or multiple state objects into another state

- In Compose, recomposition occurs each time an observed state object or composable input changes. A state object or input may be changing more often than the UI actually needs to update, leading to unnecessary recomposition.
- You should use the **derivedStateOf** function when your inputs to a composable are changing more often than you need to recompose. This often occurs when something is frequently changing, such as a scroll position, but the composable only needs to react to it once it crosses a certain threshold. **derivedStateOf** creates a new Compose state object you can observe that only updates as much as you need. In this way, it acts similarly to the Kotlin Flows `distinctUntilChanged()` operator.
- In this snippet, **firstVisibleItemIndex** changes any time the first visible item changes. As you scroll, the value becomes 0, 1, 2, 3, 4, 5, etc. However, recomposition only needs to occur if the value is greater than 0. This mismatch in update frequency means that this is a good use case for **derivedStateOf**.

```
@Composable
// When the messages parameter changes, the MessageList
// composable recomposes. derivedStateOf does not
// affect this recomposition.
fun MessageList(messages: List<Message>) {
    Box {
        val listState = rememberLazyListState()

        LazyColumn(state = listState) {
            // ...

            // Show the button if the first visible item is past
            // the first item. We use a remembered derived state to
            // minimize unnecessary compositions
            val showButton by remember {
                derivedStateOf {
                    listState.firstVisibleItemIndex > 0
                }
            }

            AnimatedVisibility(visible = showButton) {
                ScrollToTopButton()
            }
        }
    }
}
```

SideEffectsSnippe

Side Effects Best Practices

- **Use Side-Effect Functions:** Emphasize the importance of using dedicated side-effect functions such as `LaunchedEffect`, `DisposableEffect`, and `SideEffect` for handling non-composable tasks within Compose.
- **Minimize Side-Effects:** Encourage developers to minimize the number of side-effects in their Compose code by keeping side-effectful operations to a minimum and focusing on pure, composable functions whenever possible.
- **Keep Side-Effects Isolated:** Recommend isolating side-effects within dedicated side-effect functions or scopes to prevent them from interfering with the reactive and declarative nature of Compose.
- **Handle State Carefully:** Discuss strategies for handling state in conjunction with side-effects, such as ensuring proper synchronization and avoiding race conditions or conflicts.

Side Effects Conclusion

Keep in mind: “Never run any non-composable code inside composable function, always use side-effect for that”

Best Practice:

- Avoid non-composable code within composable functions.
- Utilize side-effect functions for non-composable tasks.

Why?

- Preserve Compose's reactive and declarative nature.
- Ensure performance, stability, and predictability.
- Facilitate clean and maintainable code.

Key Takeaway:

- Embrace side-effects for seamless integration of non-composable tasks in Jetpack Compose.

Compose with libraries

Compose with libraries

You can use almost the same libs as you used in traditional Android development, however there will be a few busy libs that require a slight change in approach.

- Activity
- ViewModel
- Streams of data(LiveData, Flow)
- Navigation
- Hilt
- Paging
- Map

Read more: [Compose with libraries](#)

References

- <https://fpt-software.udemy.com/course/jetpack-compose-masterclass>
- <https://developer.android.com/develop/ui/compose>
- <https://developer.android.com/codelabs/jetpack-compose-basics>
- <https://developer.android.com/codelabs/jetpack-compose-advanced-state-side-effects>
- <https://developer.android.com/codelabs/basic-android-kotlin-compose-using-state>
- <https://developer.android.com/codelabs/jetpack-compose-state>
- <https://developer.android.com/codelabs/jetpack-compose-migration>
- <https://github.com/doanpt/Android-Compose>
- <https://github.com/doanpt/ComposeLearning>

THANK YOU!

